Agentic AI System Design Report

LLM-Driven Multi-Agent Workflow with MCP Tool Server

Project 6 · Capstone · Sandip Dey

GitHub: https://github.com/sandipdey2/agentic-rag-system

Overview

This system is a small-scale agentic workflow that answers questions about a 10-fact public academic knowledge base and performs simple arithmetic. An Orchestrator implemented with an OpenAI model decides whether to call a tool, which tool to call, and how to compose the final answer. Tools are exposed through an MCP-style Tool Server. The system is intentionally bounded: it answers only the topics present in the knowledge base and basic arithmetic; it refuses general knowledge or requests that attempt to override its instructions.

System Architecture and Design

The architecture consists of an Agent Gateway, an LLM Orchestrator that performs a Plan-Act-Observe decision, specialized RAG and Calculator agents, a Guardrail agent, a Reviewer, and an MCP Tool Server. The MCP server registers tools with schemas, lists them, and sanitizes responses. Session messages and a step trace are kept in LangGraph state. An architecture diagram is provided as architecture_diagram.pdf and also rendered in the notebook as Mermaid. Components that are not claimed include a persistent long-term memory store and real rate-limit enforcement beyond the MCP call log.

Decision Logic and Behavior

The Orchestrator LLM is the sole decision maker for routing. On knowledge questions it selects the RAG tool; on arithmetic it selects the calculator; on out-of-scope or injection attempts it routes to the Guardrail. Observed examples: “What is a GAN?” returns the exact knowledge-base fact; “What is 12 * 8 + 5?” returns Result: 101; weather and prompt-injection queries are refused. The Reviewer then confirms that a grounded answer was produced.

Observed Behavior and Evaluation

On five held-out queries the system achieved 4/5 (80%). Successful cases: Variational Autoencoder (grounded fact), RAG (grounded fact), arithmetic (Result: 69), and out-of-scope refusal. The MNIST query was incorrectly refused by the LLM Orchestrator, revealing a limitation in routing on short topic names. This failure is reported honestly rather than hidden. MCP tool calls were logged for every successful tool invocation.

RAGAS-style Metrics (computed on actual answers)

Simplified local metrics inspired by RAGAS were computed on the final answers produced by the agent (not on the raw context alone). From the notebook run:

• Faithfulness: 0.67

• Answer Relevance: 0.08

• Context Precision (approx): 0.67

Faithfulness measures the fraction of answer tokens that appear in the retrieved context. Answer Relevance measures token overlap between the query and the final answer. Context Precision measures whether the retrieved document matched the query topic. These values come directly from the notebook outputs.

Safety, Reliability, and Transparency

The system prompt constrains the Orchestrator to the knowledge base and arithmetic. The calculator rejects non-arithmetic characters. Every tool call and the final answer appear in the printed trace. No API keys are stored in the repository; the key is supplied at runtime via the environment.

Ethical and Responsible Use Considerations

The system is limited to a static academic knowledge base and arithmetic. It will not provide medical, financial, or general-knowledge advice. Prompt-injection attempts are refused. The design makes every tool call and the final answer visible, supporting transparency for academic review.

Limitations and Future Improvements

Limitations include the small static knowledge base, occasional LLM routing errors on short topic names (demonstrated by the MNIST failure), and the fact that the reviewer only checks answer presence rather than full semantic faithfulness. Realistic next steps are stronger topic coverage in the routing prompt, embedding-based retrieval, and a reviewer that performs a simple entailment check against the tool output.

References

Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. Advances in Neural Information Processing Systems, 33, 9459–9474.

Wooldridge, M. (2009). An introduction to multiagent systems (2nd ed.). John Wiley & Sons.